Software Engineering Project

Matthew Presley

102202996

16.01.26

IU Berlin Campus

This document is an abstract summarizing the design and implementation of the 3Chatbots web application as well as some lessons learned in the development process. The goal of the project is to develop a lightweight evaluation tool that allows the user to submit one prompt and compare the responses of up to 3 Large Language Models in a side-by-side, real-time comparison. The project followed an agile methodology and used containerization for modularity, abstraction, and reproducibility.

The application is implemented using Django, the Python web framework. A HTML template-based frontend is used in conjunction with a Django REST backend. User prompts are submitted through the frontend and forwarded to the backend API, which calls external large language models accessed through the Hugging Face Inference API. There is a service layer to encapsulate model sessions to mitigate external API risks. Prompt sessions and responses are persisted through a SQLite database which is easily integrated due to the Django framework. Docker compose is used to orchestrate the multi-container application, using nginx for HTTPS request routing.

The functional requirements focused on performance stating users had to be allowed to enter a prompt, they could select up to 3 models, and retrieve responses through a data pipeline. Additionally, the results are to be displayed side by side. The user shall be able to enter a new prompt and the prompt will be stored in a database. The non-functional requirements focused on reponsiveness, heterogeneous broswer accessibility, and data protection. A couple examples of data protection are keeping usernames, passwords, and other user data out of system logs. Some other non-functional requirements are proper error-handling and testing.

The system architecture is seen in Figure 1 below. This models the frontend, backend, database, nginx proxy, and Hugging Face Inference API components.

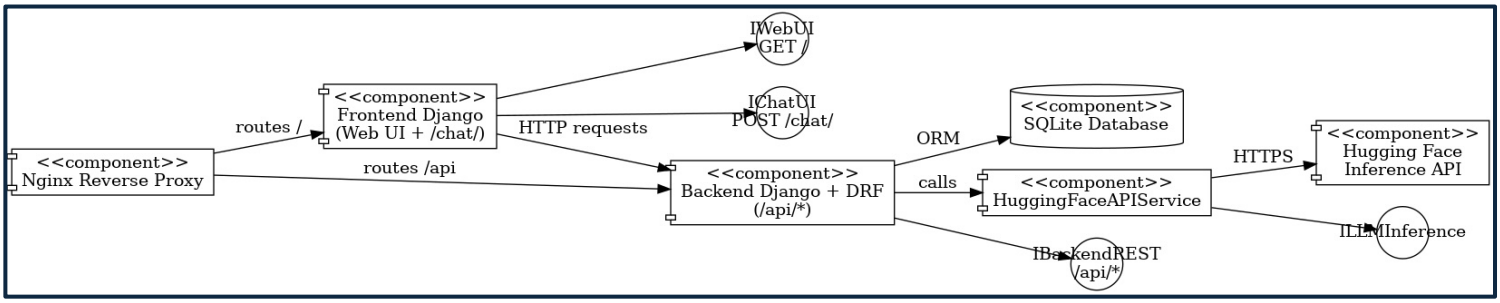

Figure 1 - UML Component Diagram

The testing strategy was done prior to deployment to AWS using manual interactive tests and API endpoint tests. Both the backend and frontend have standalone Python scripts for testing. The backend is tested by validating model retrieval, prompt submission, session persistance, and the database can be flushed to get a clean start for the application. HTTP response codes are verified as well. The frontend tests include a method to ensure both frontend and backend servers are running and available. A test is included to ensure the index routing is functioning properly. There

are also automated and manual end to end prompt-response sessions tested at the command line. Interactive tests were designed to support manual verification during development.

Regarding lessons learned, the importance of architectural modularity is evident when the application had a critical external dependance on a third-party LLM provider Hugging Face. Also encapsulation of API calls in the service layer demonstrated how easily changing providers can be to try other language models and how it can simply validation. A major lesson learned was the complexity caused by containerization. For a local deployment, containerization is challenging. For online deployment on AWS, a multi-container application introduced configuration challenges, reinfocing the need to clear and consistent documentation to reproduce deployment. Agile development allowed for integration testing early that would have been more obscure if on AWS already. Overall the project highlighted the benefits of modular design and the ability for rapid, reproducible prototyping given platforms like Docker and AWS.